Chapter 17
Mathematics Software

Abstract In this chapter we describe open-source software for mathematics. We discuss Maxima, a general purpose symbolic math software system. GNU Octave which operates on matrices and can be used for signal processing functions. Statistical computing with R and PSPP is described, as well as number theory, group theory and graph isomorphism checking using PARI, and Nauty. The open-source math software Axiom, REDUCE, Singular, CoCoA, and Macaulay are discussed in their context of algebraic geometry and commutative algebra. The Polytope analysis software 'polymake' is shown, and with the help of examples, its use in calculating polytope properties is discussed. The TeXmacs front-end and editing platform is described as well. To end the chapter we present, Sage, which is a Python interface to many of the other software mentioned in this chapter. The Computational Geometry Algorithms Library (CGAL) is described for its use in solving computational geometry and discrete geometry problems.

Contents

17.1 Maxima **358**
17.2 GNU Octave **358**
17.3 R : Data analysis and Graphics **361**
17.4 PSPP **365**
17.5 Pari **370**
17.6 Nauty **370**
17.7 Axiom **371**
17.8 Reduce **373**
17.9 Singular Computer Algebra System **375**
17.10 polymake: software to analyze Polytopes **379**
17.11 Other Math Systems **381**
17.12 CGAL (Computer Geometry Algorithms and Library) **385**
17.13 TeXMacs **387**
17.14 Sage **388**
17.15 Conclusion **390**

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_17, © Springer Science+Business Media, LLC 2011

17.1 Maxima

Maxima is an open-source symbolic math software written in Common Lisp. It is based upon Macsyma (developed in 1982 at MIT). It supports symbolic differentiation, integration and operations on polynomials.

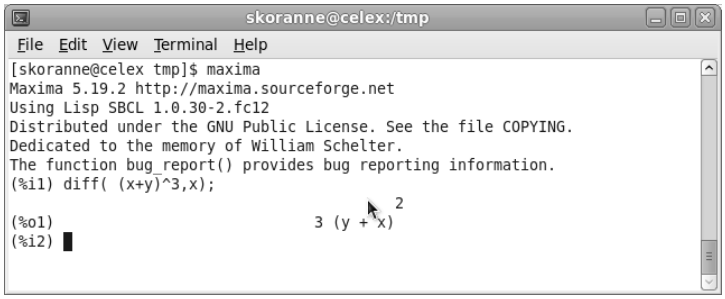


Fig. 17.1 Maxima text console

```
(%i1) u : expand((a+b)^4);
       4         3       2  2       3       4
(%o1) b  + 4 a b  + 6 a  b  + 4 a  b + a
(%i3) factor(10!);
                8  4  2
(%o3)          2  3  5  7

(%i5) integrate( (1/(1-x^2)),x);
      log(x + 1)    log(x - 1)
(%o5) ---------- - ----------
          2             2
```

17.2 GNU Octave

GNU Octave is a high-level language for manipulating matrices and performing numerical computations. The command-line interpreter is called `octave` and the software provides an interface for solving linear and non-linear problems. GNU Octave in turn is built upon many of the mathematical libraries described above (including FFTW, LAPACK and BLAS), and as such provides an uniform syntax and mechanism to use these libraries. The Octave language is mostly compatible with Matlab and indeed matrix manipulation and computation is one of the mainstays of GNU Octave's usage.

GNU Octave also has interfaces to plot graphs, load audio sample files, perform image processing (using the ImageMagick library to perform image IO). A graphical front end for Octave using Qt (called `qtoctave`) is also available (see Figure 19.6

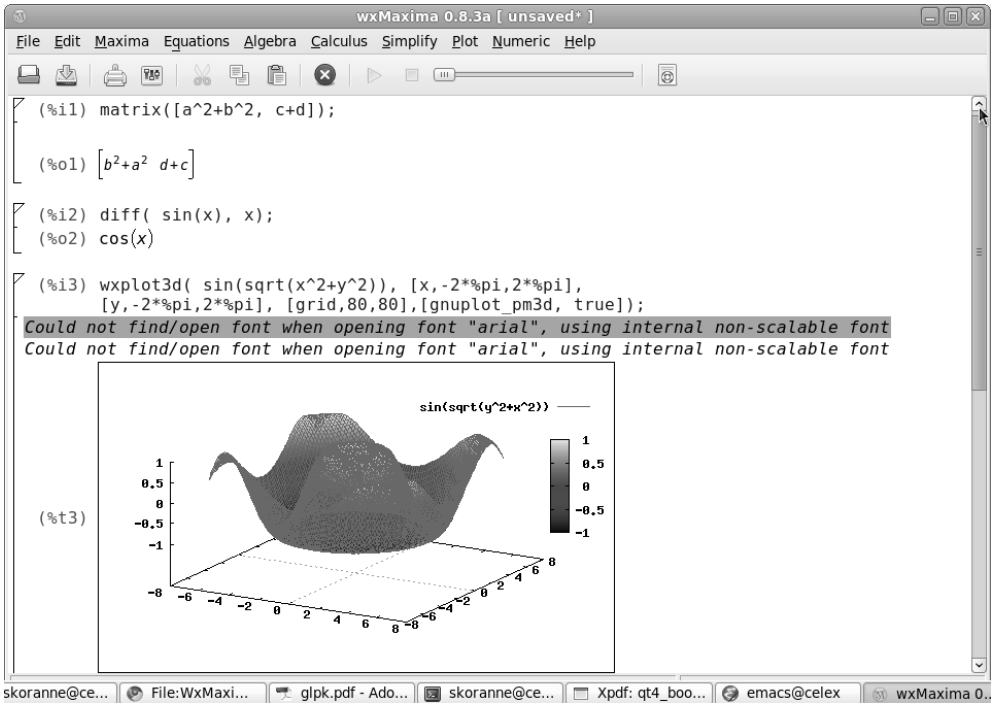


Fig. 17.2 Maxima software running in wxMaxima

for a screen-shot). In this section we describe some useful functions available in GNU Octave. To launch GNU Octave on the command-line we run `octave`:

```
octave:1> a=rand(3,3)
a =

  0.108517  0.813629  0.677108
  0.464601  0.260034  0.506348
  0.077527  0.382319  0.129084

octave:2> b=rand(3,3)
b =

  0.441143  0.096095  0.551832
  0.907029  0.146410  0.290589
  0.264928  0.842573  0.647098

octave:3> a+b
ans =

  0.54966  0.90972  1.22894
  1.37163  0.40644  0.79694
  0.34245  1.22489  0.77618

octave:4>
```

The GNU Octave command to make a matrix is shown above. A random matrix can be generated using the `rand` function. Octave supports built-in datatypes as well as user-defined data types. The built-in datatypes supported by Octave are:

1. int8,
2. uint8,
3. int16,
4. uint16,
5. int32,
6. uint32,
7. int64,
8. uint64,
9. double.

In addition, GNU Octave also supports complex and integer scalars (and matrices of the above types). String objects consist of sequence of characters. The type of the variable can be inspected by using the `typeinfo` function.

User-defined data structures can be defined as:

```
octave:1> p = struct();
octave:2> p.x = 10;
octave:3> p.y = 20;
octave:4> p
p =
{
  x =  10
  y =  20
}
```

17.2.0.1 Index expressions

A *scalar*, *row-vector* and *column-vector* can be defined as:

- `a(2)`: defines a scalar,
- `a(1:2)`: defines a row-vector,
- `a([1;2])`: defines a column vector

To create a row vector using an index expression:

```
for i = 1:4
    mat(i) = i * i
endfor
mat = 1 4 9 16
```

GNU Octave can return multiple values from functions. Octave supports arithmetic operators, comparison operators, and Boolean operators. In addition to using the interpreter, Octave programs can be stored in a file. Consider a program as shown in Listing 17.1 which defines a function `hello_world`.

```
# GNU Octave file

function hello_world(name)
  printf (mfilename("fullpath") );
  printf ("\nHello from %s\n", name );
endfunction
```

Listing 17.1 GNU Octave program script

When the path in which this file is placed located in the Octave search path (a path can be added to the Octave search path using the `addpath` function), the function `hello_world` is available to use in the interpreter. One thing to keep in mind is that the function should not be called inside the file (otherwise it results in infinite recursion). To prevent this, place an expression which not a function as the first non-comment token in the file. Using the `mfilename` we can print the name of the current file (using the "fullpath") argument to `mfilename`, we can print the full path to the file. Any function which is not named the same as the file is a function which is only visible inside the file and cannot be called from outside. Other important functionality of Octave are listed below:

1. Input/output functions,
2. Plotting,
3. Matrices:
   - Diagonal matrix
   - Sparse matrix
4. Linear algebra,
5. Non-linear equation,
6. Numerical integration,
7. Differential equations,
8. Optimization,
9. Statistics,
10. Polynomial manipulations,
11. Signal processing:
    - Audio processing,
    - Image processing.

17.3 R : A Programming Environment for Data Analysis and Graphics

R is a programming language and software environment for statistical computing and graphics. It is an implementation of the S programming language with lexical scoping semantics inspired by Scheme (see Section 1.6.9). The R language has become standard among statisticians for the development of statistical software, and

is widely used for statistical software development and data analysis. Amongst its many features are:

1. Operators for calculation on arrays,
2. Simple but effective programming language 'S',
3. Tools for data analysis,
4. Effective data I/O and storage,
5. Integrated graphical display and plotting.

R provides a wide variety of statistical (including linear and nonlinear modeling, classical statistical tests, time-series analysis, classification, and clustering, etc.) and graphical techniques. Consider a small transcript of an R session below:

```
> x <- c(1,3,5,7,11,13)
> y <- x^2
> mean(y)
[1] 62.33333
> var(y)
[1] 4599.467
> summary(lm(y ~ x))

Call:
lm(formula = y ~ x)

Residuals:
      1       2       3       4       5       6
 19.478  -1.043 -13.565 -18.087  -3.130  16.348

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   -32.74      13.03  -2.513 0.065821 .
x              14.26       1.65   8.644 0.000985 ***
---
Signif. codes:  0 *** 0.001 ** 0.01 * 0.05 . 0.1   1

Residual standard error: 17.09 on 4 degrees of freedom
Multiple R-squared: 0.9492,Adjusted R-squared: 0.9365
F-statistic: 74.71 on 1 and 4 DF,  p-value: 0.0009853
```

In the next section we shall describe another software system for statistical analysis, PSPP see Section 17.4. R, although used for statistical analysis is designed to be a flexible environment which supports statistical analysis. Moreover, in R the computed results of operations are stored in objects on which subsequent analysis can be performed. PSPP is designed to produce output immediately upon computation. Some of the useful command in R are given below:

```
> q() # quit R
> ?solve # give help on solve function
> example(solve) # show an example of the solve function
> plot( x <- rnorm(64), type="s", main="Title")
> log10(2)
[1] 0.30103
> sqrt(100)
```

```
[1] 10
> exp(1.01)
[1] 2.745601
> source( "file.R") # read command from file
> sink( "file.log") # write output to file
> objects() # data persistence
> ls()
"A" h8" "hilbert"
> ls
<environment: namespace:base>
> c(1,1,2,3)  # construct a vector of 1,1,2,3
[1] 1 1 2 3
> x <- c(1,1,2,3) # assign this vector to var. x
> x
[1] 1 1 2 3
> c(x,0,x) -> y # assignment in the other direction
> x[1] = 8 # vectors in x start at 1
> x        # x is now modified
[1] 8 1 2 3
> y        # but y is not, vectors are copied in assignment
[1] 1 1 2 3 0 1 1 2 3
> sum(x)
[1] 14
> mean(x)
[1] 3.5
> x <- c(1:10)
[1]  1  2  3  4  5  6  7  8  9 10
> x <- c(1,2,...,8)
seq(from=1,to=10, length=3) # functions in R can have named params
[1]  1.0  5.5 10.0
> logical_vector <- x > 2 # vector of booleans if x > 2
m <- array(23:43, dim=c(4,4))
> m
     [,1] [,2] [,3] [,4]
[1,]   23   27   31   35
[2,]   24   28   32   36
[3,]   25   29   33   37
[4,]   26   30   34   38
> eigen(m)
$values
[1]  1.226523e+02+0.000000e+00i -6.522506e-01+0.000000e+00i
[3]  2.949706e-15+1.401517e-15i  2.949706e-15-1.401517e-15i

$vectors
              [,1]          [,2] ...
[1,] -0.4752248+0i  0.7359579+0i ...
[2,] -0.4915202+0i  0.2934262+0i ...
[3,] -0.5078157+0i -0.1491055+0i ...
[4,] -0.5241111+0i -0.5916372+0i ...

m_transpose = aperm(m, c(2,1)) # transpose is a permutation
> m_transpose
     [,1] [,2] [,3] [,4]
[1,]   23   24   25   26
```

```
[2,]   27   28   29   30
[3,]   31   32   33   34
[4,]   35   36   37   38
>
```

As can be seen above, there is a difference between calling a function `ls()` versus the object representing the function `ls`. The former calls the `ls` function, while the latter returns the definition of the function. Vectors can be assigned to other vectors using the `c()` function. Vectors are copied by value and not by reference. Matrix examples are shown as well; also see Section 17.2 for GNU Octave for matrix computations.

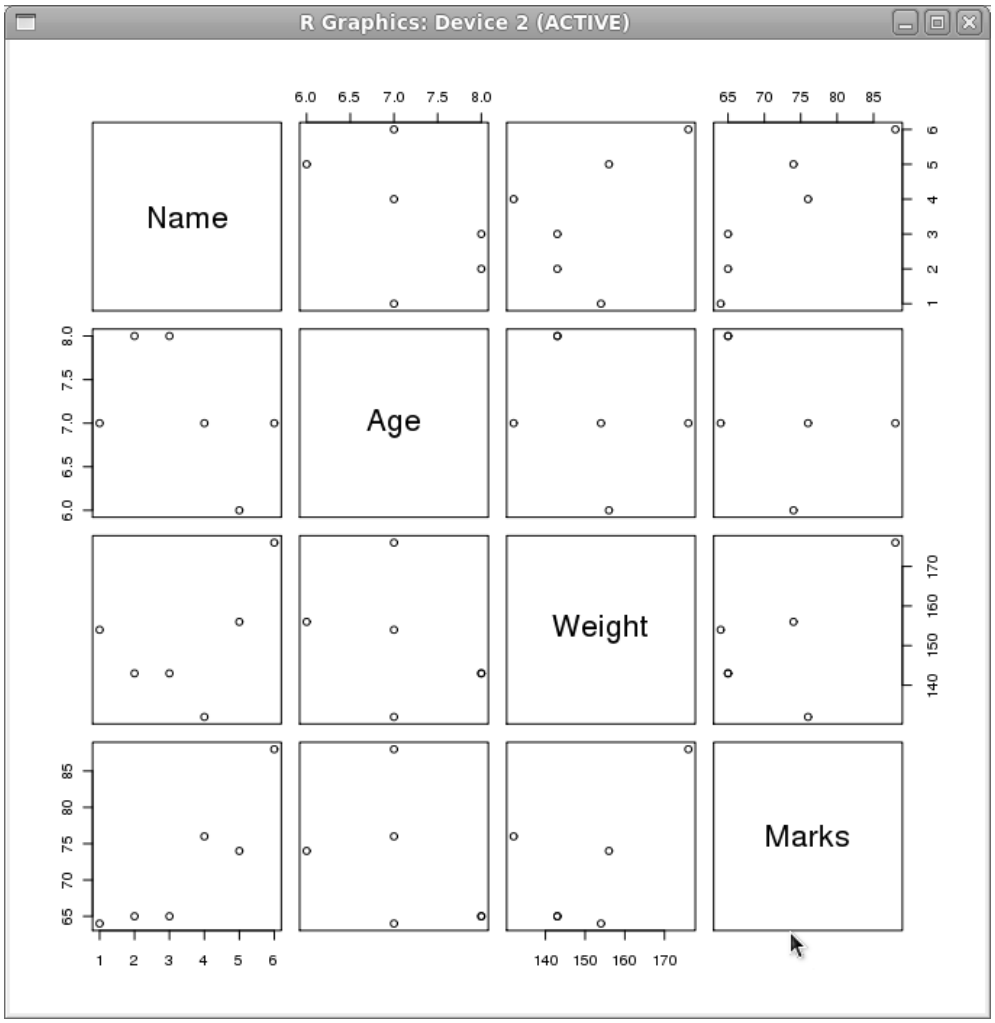


Fig. 17.3 Data summary plot using R

Data present in external file can be read using the `read.table` function. The data file should have a header line defining the name of the variable present in each subsequent row. The data is read into a *data-frame*.

```
roll_table <- read.table("rollcall.data", header=TRUE)
> roll_table
    Name Age Weight Marks
1   Jack   8    143    65
2   Jill   8    143    65
...
> sum(roll_table["Age"])
[1] 43
mean(roll_table["Age"])
     Age
7.166667
> mean( roll_table["Marks"])
Marks
   72
>summary(roll_table["Marks"])
     Marks
 Min.   :64.0
 1st Qu.:65.0
 Median :69.5
 Mean   :72.0
 3rd Qu.:75.5
 Max.   :88.0
> plot( roll_table)
```

A complete summary of this data can be generated using the `plot` function as shown in Figure 17.3. In addition to the built-in functions, R also has a complete programming language in which users can define their own functions. The programming language supports the usual paradigms of looping constructs, **`if`**-then-**`else`** and object-oriented programming.

```
> my_sum <- function(a,b) { a+b }
> my_sum(10,20)
[1] 30
```

17.4 PSPP

PSPP is a program for statistical analysis of sampled data. It is designed to be an open-source alternative of other proprietary statistical analysis programs. PSPP can be used to perform descriptive statistics, T-tests, non-parametric tests, linear regressions alongwith the basic analysis of frequency, mean, median mode on variable data. Some of the distinctive features of PSPP are:

- Portability: available as open-source software on many operating systems including GNU/Linux,
- Has both command-line and GUI interface,
- High performance: can support billion cases and variables,
- Compatibility: data and syntax are compatible with other statistical analysis programs,

- Plotting: can produce output directly in Postscript,
- Easy import of data: from comma-separated files, spreadsheets, and databases.

A sample dataset can be managed by creating a data view containing columns of data types as shown in Figure 17.4.

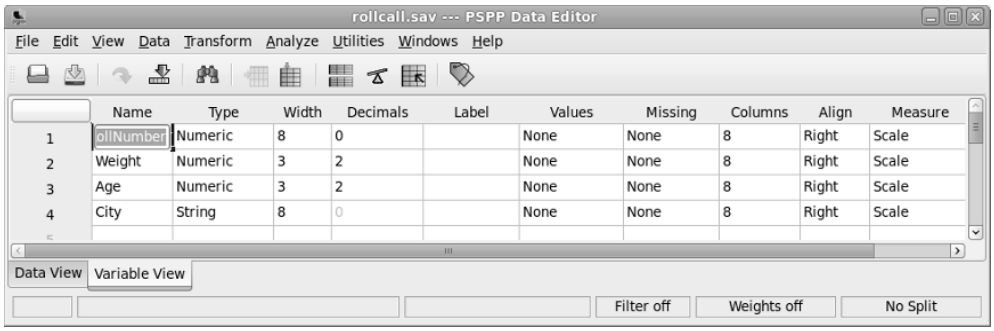


Fig. 17.4 Variable view in PSPP

PSPP (in batch mode) reads a syntax file, and a given data file, performs the analysis and writes the output to a listing file (or standard output). The output tables and charts can be in ASCII or Postscript and HTML. In batch mode `pspp` accepts various command-line arguments specifying the input file, output file, and other options. For graphical user interface the command `psppire`. An example session with PSPP to analyze data is shown in Figure 17.5. PSPP is designed for statistical anal-

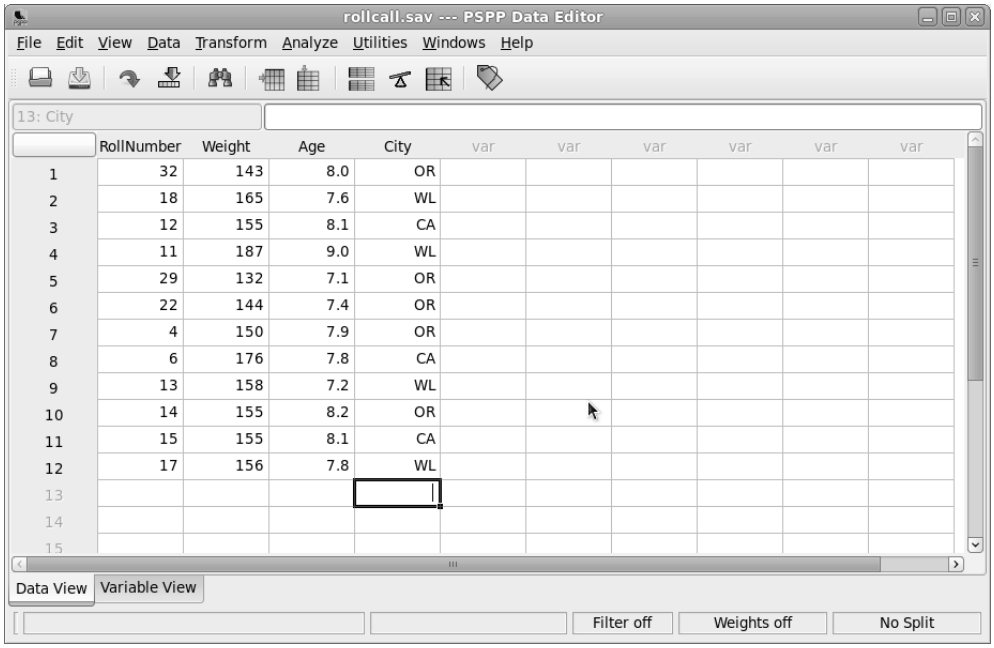


Fig. 17.5 PSPP software to analyze data

ysis of data, thus data input/output is central to the program. Each datum represents (or belongs to) a *case*, also called an observation. In Figure 17.5 12 observations are listed. The data can also be read from TEXT files, PostgreSQL databases, and GNUmeric spreadsheets. Consider an example to read the UNIX `/etc/passwd` file:

```
GET DATA /TYPE=TXT /FILE='/etc/passwd' /DELIMITERS=':'
         /VARIABLES=username A20
                    password A40
                    uid F10
                    gid F10
                    ...
                    shell A40
```

Additionally the data can be included as part of the program for simple analysis. Consider the example given below:

```
DATA LIST /X 1-3
BEGIN DATA.
109
2
65
4
432
10
15
20
198
24
12
END DATA.
COMPUTE X=X/2.
DESCRIPTIVES X.
```

This PSPP program performs a division of 2 on the sample data before computing the statistical properties. Running `pspp <file>` gives the analyzed listing as:

```
1.1 DATA LIST.  Reading 1 record from INLINE.
+--------+------+-------+------+
|Variable|Record|Columns|Format|
#========#======#=======#======#
|X       |     1|  1-  3|F3.0  |
+--------+------+-------+------+

2.1 DESCRIPTIVES.  Valid cases = 11;
        cases with missing value(s) = 0.
+--------#--+-----+-------+-------+-------+
|Variable# N| Mean|Std Dev|Minimum|Maximum|
#========#==#=====#=======#=======#=======#
|X       #11|40.50|  65.43|   1.00| 216.00|
+--------#--+-----+-------+-------+-------+
```

Another example using statistical regression with formatted input is shown below:

```
echo "Rollcall in PSPP"
DATA LIST /Name 1-10 (A) Age 11-12 (0) Weight 13-15 (0)
                         Marks 16-17 (0)
BEGIN DATA.
Jack        814365
Jill        814365
John        713276
Mark        615674
Daniel M    715464
Zack, Paul 717688
END DATA.
LIST.
REGRESSION /VARIABLES=Age Marks /STATISTICS DEFAULTS
           /DEPENDENT=Marks.
REGRESSION /VARIABLES=Age Weight /STATISTICS DEFAULTS
           /DEPENDENT=Weight.
REGRESSION /VARIABLES=Weight Marks /STATISTICS DEFAULTS
           /DEPENDENT=Marks.
```

The output of this program is:

```
Rollcall in PSPP

1.1 DATA LIST.  Reading 1 record from INLINE.
+--------+------+-------+------+
|Variable|Record|Columns|Format|
#========#======#=======#======#
|Name    |     1|  1- 10|A10   |
|Age     |     1| 11- 12|F2.0  |
|Weight  |     1| 13- 15|F3.0  |
|Marks   |     1| 16- 17|F2.0  |
+--------+------+-------+------+

      Name Age Weight Marks
---------- --- ------ -----
Jack         8    143    65
Jill         8    143    65
John         7    132    76
Mark         6    156    74
Daniel N     7    154    64
Zack, Paul   7    176    88

2.1 REGRESSION.  Model Summary
#============#========#================#=========================#
#          R #R Square|Adjusted R Square|Std. Error of the Estimate#
#========#===#========#================#=========================#
#        |.45#     .21|             .21|                     9.32#
#========#===#========#================#=========================#

2.2 REGRESSION.  ANOVA
#==================#==============#==#===========#====#============#
#                   #Sum of Squares|df|Mean Square|  F |Significance#
#========#==========#==============#==#===========#====#============#
#        |Regression#         90.35| 1|      90.35|1.04|         .37#
#        |Residual  #        347.65| 4|      86.91|    |            #
#        |Total     #        438.00| 5|           |    |            #
#========#==========#==============#==#===========#====#============#

2.3 REGRESSION.  Coefficients
#==================#======#==========#====#=====#============#
#                   #   B  |Std. Error|Beta|  t  |Significance#
#========#==========#======#==========#====#=====#============#
#        |(Constant)#112.47|     39.87| .00| 2.82|         .22#
#        |    Age   # -5.65|      5.54|-.45|-1.02|         .35#
#        |          #      |          |    |     |            #
#========#==========#======#==========#====#=====#============#

3.1 REGRESSION.  Model Summary
#============#========#================#=========================#
#          R #R Square|Adjusted R Square|Std. Error of the Estimate#
#========#===#========#================#=========================#
#        |.36#     .13|             .13|                    15.78#
#========#===#========#================#=========================#
```

Using PSPP we can easily compute the dependence of variables on others and compute their statistical relationships. PSPP includes several statistical analysis functions, as shown in Figure 17.6.

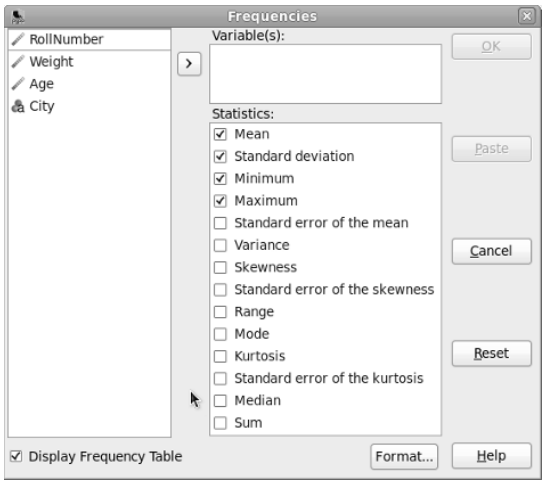


Fig. 17.6 Statistical analysis function in PSPP

When the data is analyzed PSPP produces a detailed report as given below:

3.1 FREQUENCIES. Weight

Label	Value	Frequency	Percent	Valid Percent	Cum Percent
	132	1	8.33	8.33	8.33
	143	1	8.33	8.33	16.67
	144	1	8.33	8.33	25.00
	150	1	8.33	8.33	33.33
	155	3	25.00	25.00	58.33
	156	1	8.33	8.33	66.67
	158	1	8.33	8.33	75.00
	165	1	8.33	8.33	83.33
	176	1	8.33	8.33	91.67
	187	1	8.33	8.33	100.00
Total		12	100.0	100.0	

N	Valid	12	Range	55.00
	Missing	0	Minimum	132.00
Mean		156.33	Maximum	187.00
Mode		155.00	50 (Median)	156
Std Dev		14.69		
S.E. Kurt		1.23		
S.E. Skew		.64		

17.5 Pari

PARI/GP is a mathematical software system for performing research in number theory and group theory. Running `gp` on GNU/Linux gives:

```
[skoranne@celex tmp]$ gp
            GP/PARI CALCULATOR Version 2.3.4 (released)
     i686 running linux (ix86/GMP-4.3.1 kernel) 32-bit version
compiled: Oct 20 2009, gcc-4.4.2 20091018 (Red Hat 4.4.2-4) (GCC)
         (readline v6.0 enabled, extended help available)

               Copyright (C) 2000-2006 The PARI Group

PARI/GP is free software, covered by the GNU General
Public License, and comes WITHOUT ANY WARRANTY WHATSOEVER.

Type ? for help, \q to quit.
Type ?12 for how to get moral (and possibly technical) support.

parisize = 4000000, primelimit = 500000
?
```

The software comprises of two components:

1. PARI: is the underlying C library which implements the required functions for operating on groups,
2. gp: is a command line interface to the above library. It is structured as a calculator which interfaces to the user.

PARI/GP supports arbitrary precision computation on integers and thus can easily perform factorial and prime number computations.

17.6 Nauty

Nauty is a computer program and library for computing automorphism group of graphs and digraphs. It can also be used to produce a canonical labeling of the vertices of the graph. Nauty is written in C language and is thus easily portable. For graphs which can fit in the wordsize of the machine (32-bit or 64-bit) it has excellent performance. Nauty can be used as a library, or it can be used as a graph isomorphism checker. The command-line program `dreadnaut` which ships with Nauty provides orbit (automorphism group of the vertices of the graph) calculation. Consider the 3-dimensioned simplex with 4 vertices. This is a complete graph (every vertex is connected to every other vertex). See Figure 17.7.

```
Dreadnaut version 2.4 (32 bits).
> n=4
> g
 0 : 1 2 3;
```

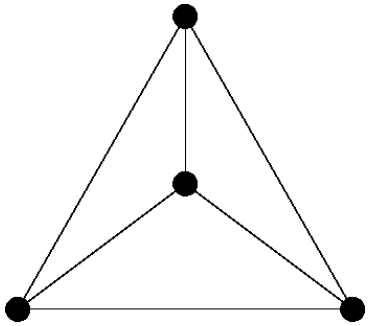


Fig. 17.7 Graph of 3-simplex

```
 1 : 2 3;
 2 : 3;
 3 : .
> x
(2 3)
level 3:  3 orbits; 2 fixed; index 2
(1 2)
level 2:  2 orbits; 1 fixed; index 3
(0 1)
level 1:  1 orbit; 0 fixed; index 4
1 orbit; grpsize=24; 3 gens; 10 nodes; maxlev=4
tctotal=16; cpu time = 0.00 seconds
```

Using `dreadnaut` we first specify that the graph has 4 vertices, then we input the adjacency of the graph. The 'x' command runs the canonical labeling computation which prints the automorphism groups (orbits). The graph is completely symmetrical and thus there is only 1 orbit.

17.7 Axiom

OpenAxiom is an open-source symbolic math software. However, unlike Maxima and other symbolic math software Axiom has been designed for conducting research in mathematics. It has an integrated environment, compiler, large set of mathematical libraries. In text mode Axiom returns:

```
         AXIOM Computer Algebra System
          Version: Axiom (March 2008)
 Timestamp: Friday April 25, 2008 at 17:38:07

 Issue )copyright to view copyright notices.
 Issue )summary for a summary of useful system commands.
 Issue )quit to leave AXIOM and return to shell.


   Re-reading compress.daase   Re-reading interp.daase
```

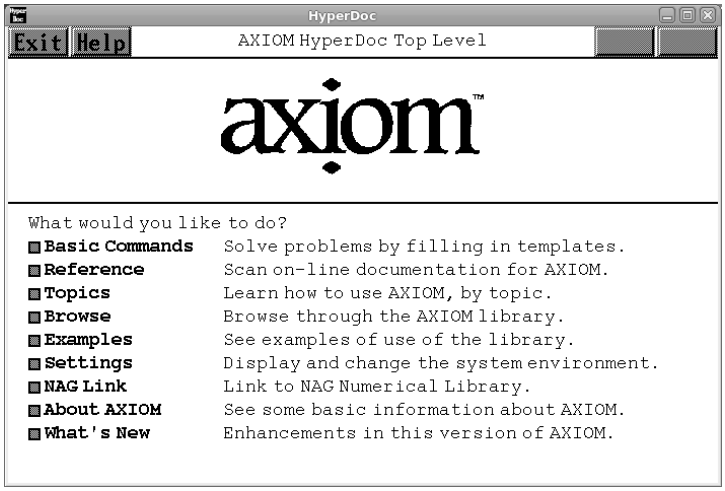


Fig. 17.8 AXIOM software

```
   Re-reading operation.daase
   Re-reading category.daase
   Re-reading browse.daase
(1) -> integrate(1/(x**3 * (a+b*x)),x)
         2 2                  2 2                    2
      - 2b x log(b x + a) + 2b x log(x) + 2a b x - a
(1)   ---------------------------------------------
                             3 2
                           2a x
```

The above example shows the symbolic math capabilities of Axiom. However, the most powerful feature of Axiom lies in its design and programming languages (SPAD and Aldor). Axiom is type aware, not only in the programming language sense, but also mathematically. Using *Categories* which define algebraic properties mathematical correctness is ensured. Categories allow the algorithms implemented in Axiom's programming language to be defined in their natural settings; Axiom's interpreter constructs the domain based on user input as shown below:

```
  (5)  5
          Type: PositiveInteger
(6) -> (x+y)**2
(6) ->
          2           2
   (6)  y  + 2x y + x
         Type: Polynomial Integer

-> M = [ [ x**2+1, 0], [0, x]]::Matrix( POLY(FRAC(INT)))
    +1  0+ 2   +0  0+    +1  0+
 M= |    |x  + |    |x + |    |
    +0  0+     +0  1+    +0  0+
Type: Equation Polynomial SquareMatrix(2,Fraction Integer)
```

Axiom is aware of the type (mathematical type) of the input entered into the interpreter. We can ask Axiom for *type conversion*:

```
(10) -> (22/7)::Float
(10) ->

   (10)  3.1428571428 571428571
                Type: Float

(11) -> %::Fraction Integer

(11) ->
           22
   (11)    --
            7
                Type: Fraction Integer
```

Axiom also supports complex numbers as well as arbitrary long integers. To quit the interpreter, enter:

```
-> )quit
```

17.8 Reduce

Reduce is an open-source interactive math software for general algebraic computations. It is written in Lisp (can be compiled using Codemist Standard Lisp (csl) or SLisp). It supports the following functionality:

1. Expansion and ordering of polynomials,
2. Automatic simplification,
3. Calculation on symbolic matrices,
4. Ability to add new functions using Lisp,
5. Polynomial factorization,
6. Analytic differentiation and integration.

Reduce is available as open-source and is portable, being able to run on many computer systems from GNU/Linux personal computers to supercomputers. Moreover, Reduce has built up a number of useful packages. Many of these add significant functionality such as: (i) tensor manipulation, (ii) Boolean algebra, (iii) Groebner basis and commutative algebra, (iv) overdetermined systems of partial differential equations, and (v) polynomial ideals. Invoking Reduce we get:

```
REDUCE, 15-Sep-08 ...

1: (a+b)^3;
```

```
 3      2          2    3
a  + 3*a *b + 3*a*b  + b
2: m := mat((a,b),(c,d));

     [a  b]
m := [    ]
     [c  d]


3: m*m;

[ 2                   ]
[a  + b*c   b*(a + d)]
[                     ]
[                   2 ]
[c*(a + d)  b*c + d   ]
```

Interaction with Reduce can be done completely through the interpreter, or programs can be written in Reduce which can use the underlying Lisp system for efficiency, eg:

```
7: first {a,b,c};
a
8: rest {a,b,c};
{b,c}
9: reverse {a,b,c};
{c,b,a}
```

In this sense, Reduce can be used as a special purpose Lisp system which has significant mathematical library support built into it. Reduce has a number of mathematical functions already built into the system and new ones defined by the user can be compiled and brought into the running image.

```
11: solve( x^2-8*x+1, x);
{x=sqrt(15) + 4,
 x= - sqrt(15) + 4}
```

Reduce can also generate FORTRAN statements which implement and evaluate the computation:

```
COMMENT Switch on Fortran mode
on fort;
x := (a+b+c)^2;

      x=a**2+2.0*a*b+2.0*a*c+b**2+2.0*b*c+c**2
off fort;
```

In this example, Reduce has printed the FORTRAN statement to calculate value of x from a, b, c using the formula we entered in the interpreter. This facility can be used to generate FORTRAN code for simulations, and can be compiled and run on HPC machines.

17.9 Singular Computer Algebra System

Singular is an open-source computer algebra system for polynomial computation. It was designed to solve problems in commutative algebra, algebraic geometry and singularity theory. Singular algebra system features one of the fastest and most general polynomial implementation and has been used to implement Buchberger's algorithm for Groebner basis. Its main computational objects are ideals and modules over baserings (which are polynomial rings or localization of polynomial rings over fields including finite fields, rationals, floats, and algebraic extensions).

Functions included in Singular include polynomial operations, factorization, resultant, characteristic set and GCD computation. Singular has an interactive shell and a C-like programming language. Using the programming language, users have contributed extensions and application software which is now available with the Singular distribution. This is also an intended consequence of the open-source methodology.

Singular is invoked on GNU/Linux system as:

```
$./Singular/3-1-1/ix86-Linux/Singular
> 12 + 32;
44
> int x = _;
> x;
44
> int y = 12; // this is a comment
> help factorial; // launches the help in browser
```

In the above example, we can see the Singular facilities for arithmetic, variable assignment, and text comments. The last displayed result (non-assigned) is stored in an internal variable _, and variable assignment is done using the = operator. All variables in Singular must be typed, in the above assignment we have used the integer type. To use a function which is defined in a library we have to use:

```
LIB "general.lib";
// Singular proceeds to load general.lib and dependencies
> factorial(10);
3628800
> binomial(5,3);
10> kmemory;
// libname  : general.lib
// procname : kmemory
// type     : singular
```

```
> kmemory();
206
```

The "general.lib" library has many useful functions including binomial, fibonacci, exponential, prime number, sorting, factorization and more. As is shown in the above example, typing the name of the function `kmemory` returns information about the type of the variable, and we can indeed call the function using `kmemory()` which returns the number of bytes used by Singular thus far (in KB).

To perform polynomial operations in Singular we first have to define a Ring, then create a polynomial in that ring:

```
> ring R = 0, (x,y),dp;
> poly Q = (2x+y);
> Q;
2x+y
> Q*Q;
4x2+4xy+y2
> poly P = ((x+y)^2)*(x2+y5);
> P
. ;
y7+24y6+144y5+x2y2+24x2y+144x2
> factorize(P);
[1]:
   _[1]=1
   _[2]=y5+x2
   _[3]=y+12
[2]:
   1,1,2
```

The definition of a ring in Singular involves setting the ground field (0 in the above example), the names of the ring variables (x, y), and the monomial ordering to be used (dp stands for degree reverse lexicographical ordering). The notation for polynomials in Singular is coefficient, variable, power. Factorization of polynomials can also be computed with a time constraint using `timeFactorize` which accepts a second argument (number of seconds); if the factorization does not complete in the stipulated time the polynomial is considered irreducible. A field on a characteristic can be defined as:

```
> ring PZ7 = 7,(x(1..10)),ds;
> poly ZQ = (x(1)+x(2))^5 + (x(6)+x(4))^3;
> ZQ;
x(4)^3+3*x(4)^2*x(6)+3*x(4)*x(6)^2+x(6)^3+
x(1)^5-2*x(1)^4*x(2)+
3*x(1)^3*x(2)^2+3*x(1)^2*x(2)^3-2*x(1)*x(2)^4+x(2)^5
> factorize(ZQ);
[1]:
   _[1]=1
```

```
   _[2]=x(4)^3+3*x(4)^2*x(6)+3*x(4)*x(6)^2+
        x(6)^3+x(1)^5-2*x(1)^4*x(2)+
        3*x(1)^3*x(2)^2+3*x(1)^2*x(2)^3-2*x(1)*x(2)^4+
        x(2)^5
[2]:
   1,1
> poly ZQ = (x(1)+x(2))^5;
// ** redefining ZQ **
> factorize(ZQ);
[1]:
   _[1]=1
   _[2]=x(1)+x(2)
[2]:
   1,5
```

Rings can also be defined on complex numbers:

```
> ring cpz = (complex,2,i),(a,b),lp;
> cpz;
//   characteristic : 0 (complex:6 digits, additional 6 digits)
//   1 parameter    : i
//   minpoly        : (i^2+1)
//   number of vars : 2
//        block   1 : ordering lp
//                  : names    a b
//        block   2 : ordering C
> poly CQ = a2 + 3*i;
> CQ;
a2+(i*3)
> CQ*CQ;
a4+(i*6)*a2-9
// going back to a previous ring
> setring PZ7;
> PZ7;
//   characteristic : 7
//   number of vars : 10
//        block   1 : ordering ds
//                  : names    x(1).. x(10)
//        block   2 : ordering C
>
```

Matrices can be constructed in row-major order:

```
> intmat M0[3][3] = 1,2,3,4,5,6,7,8,9;
> M0;
1,2,3,
4,5,6,
7,8,9
> int j;
> int M0_trace;
> for( j=1; j <= 3; j++ )
```

```
  { M0_trace = M0_trace + M0[j,j]; }
> M0_trace;
15
```

One of the most common use of Singular is performing computations on *ideals* and *varieties* generated by a system of polynomial equations. Consider the following example:

```
> ring r = 0,(x,y,z),dp;
> poly f =  x3+y3+(x-y)*x2y2+z2;
> poly g = f^2 * (2x-y);
> ideal I = f,g;
> ideal J = subst(I, var(1),1);
> J;
J[1]=y2+z2+1
J[2]=-y5-2y3z2-yz4+2y4+4y2z2+2z4-2y3-2yz2+4y2+4z2-y+2
> J = subst(J,var(2),2);
> J;
J[1]=z2+5
J[2]=0
// Compyute ideal using Groebner basis
> ideal sI = groebner(f);
> sI;
sI[1]=x3y2-x2y3+x3+y3+z2
> reduce(g,sI);
0
// Compute the Jacobian ideal
> ideal J = jacob(f);
// ** redefining J **
> J;
J[1]=3x2y2-2xy3+3x2
J[2]=2x3y-3x2y2+3y2
J[3]=2z
```

Groëbner basis computations are very useful in many scientific disciplines are the subject of active research (many using Singular). New functions (procedures) can be defined in Singular as:

```
> proc mysize (poly Q) { return (size(Q)); }
> mysize(f);
5
>
```

Singular's procedure syntax is similar to other programming languages such as C.

17.10 polymake: software to analyze Polytopes

Polymake is an open-source math software to study, generate and operate on the combinatorics and geometry of convex polytopes and polyhedra. Polymake includes a number of tools and methods which can be used to perform experiments on polytopes. Consider the following example:

```
$ cube
usage: cube <file> <dimension> [<scale>]
$ simplex
usage: simplex <file> <dimension> [<scale>]
$ simplex 3simplex.poly 3
```

The file '3simplex.poly' is shown below:

```
_application polytope
_version 2.3
_type RationalPolytope

AMBIENT_DIM
3

DIM
3

VERTICES
1 0 0 0
1 1 0 0
1 0 1 0
1 0 0 1

N_VERTICES
4

SIMPLICIALITY
3

BOUNDED
1

CENTERED
0
```

The above example has generated the 3-simplex as shown in Figure 17.7. At this point the polytope is represented by its coordinates as shown in the Polymake section titled `VERTICES`. Polymake (upto version 2.3) operates on polytopes which

are present in files. Polymake version 2.9.7 has moved to a Perl based frontend. Polytope properties can be queried using Polymake as shown below:

```
$ polymake 3simplex.poly DIM
DIM
3
$ polymake 3simplex.poly DIAMETER
DIAMETER
1
$ polymake 3simplex.poly F_VECTOR
F_VECTOR
4 6 4
$ polymake 3simplex.poly G_VECTOR
G_VECTOR
1 0
$ polymake 3simplex.poly H_VECTOR
H_VECTOR
1 1 1 1
$ polymake 3simplex.poly CD_INDEX
CD_INDEX
c^3 + 2cd + 2dc
$ truncation -h
usage: truncation <out_file> <in_file>
       { <vertex> [ <vertex> ... ] | all }
       [ -cutoff <cf> | -noc ] [ -relabel ]
$ truncation 3simplex_0.poly 3simplex.poly 0
$ polymake 3simplex_0.poly GRAPH
GRAPH
{1 2 3}
{0 2 4}
{0 1 5}
{0 4 5}
{1 3 5}
{2 3 4}
```

The examples shown above compute certain properties of the 3-simplex. The F_VECTOR, G_VECTOR and H_VECTOR are combinatorial flag vectors computed from the face-lattice of the polytope. The CD-index encodes the face inclusion of the lattice. The last operation *truncation* produces a new polytope from the old one by truncating (removing) the specified vertex. Polymake can also be used for interactive viewing of polytopes. Version 2.9.7 and onwards have an integrated Perl frontend which can be used to perform computations on the polytope. Consider the following example:

```
$polymake
polytope > $cube = load("3cube.poly");

polytope>print"cube has dimension=",$cube->DIM,"\n";
cube has dimension = 3
polytope > print "cube graph = ", $cube->GRAPH,"\n";
cube graph = Polymake::graph::
       Graph__Undirected::__as__Polytope__GRAPH=
```

```
        ARRAY(0xacdb34c
polytope >
```

The Perl language's feature can be used alongwith Polymake's polytope data structures and functions.

17.11 Other Math Systems

In addition to the math software we have described above, there are a number of other specialized mathematics software which we briefly mention below:

17.11.1 Macaulay 2

Macaulay 2 is an open-source mathematics computer system for computation on commutative algebra and algebraic geometry. On GNU/Linux, Macaulay2 can be started using the command-line `M2`:

```
$M2
Macaulay2, version 1.3.1
with packages: ConwayPolynomials, Elimination,
               IntegralClosure, LLLBases,
               PrimaryDecomposition, ReesAlgebra,
               SchurRings, TangentCone

i1 : 2+3;
i2 : o1
o2 = 5
i3 : 10!
o3 = 3628800
i4 : 22/7 + 3/11

     263
o4 = ---
      77
o4 : QQ
```

The last example shows that rational (QQ) numbers are supported in Macaulay2. A ring can be created in Macaulay2 as follows:

```
i1 : R = QQ[x,y,z]
o1 = R
o1 : PolynomialRing
i2 : (x+y+z)^3
```

```
      3     2         2    3     2
o2 = x  + 3x y + 3x*y  + y  + 3x z +

                2         2        2    3
     6x*y*z + 3y z + 3x*z  + 3y*z  + z
o2 : R
i6 : c = matrix { { x^2 + 1, y^2, z^2+1}}
o6 = | x2+1 y2 z2+1 |

              1       3
o6 : Matrix R  <--- R
i7 : M = coker b
o7 = cokernel | x y z |
                                 1
o7 : R-module, quotient of R
```

17.11.1.1 Algebraic Geometry with Macaulay2

The most common use of Macaulay2 is studying geometric objects in (complex) affine spaces. We can define a Ring as above and construct ideals from polynomial equations as:

```
i3 : curve = ideal( x^2+y^2, x-2*y )

             2    2
o3 = ideal (x  + y , x - 2y)

o3 : Ideal of R

i4 : gb curve

o4 = GroebnerBasis[status: done;
     S-pairs encountered up to degree 1]

o4 : GroebnerBasis

i5 : o4

i6 : dim curve

o6 = 1

i7 : codim curve

o7 = 2

i8 : degree curve

o8 = 2
```

Above we construct the ideal of the curve defined by the polynomials x^2+y^2 and $x-2y$, the dimension of this curve is 1, while the co-dim and degree are 2. Recall, that the degree of a curve is the number of intersections of the curve with a general plane. We can calculate the intersection of this curve with a *surface* as:

```
i9 : surface = ideal(x+y-2)

o9 = ideal(x + y - 2)

o9 : Ideal of R

i12 : find_point = intersect(curve,surface)

                2             2
o12 = ideal (3x  - 3x*y - 6y  - 6x + 12y,
      ---------------------------------
           2          2     3      2
      - 18x y - 27x*y  - 9y  + 12x  + 24x*y
      -------------------------------------
          2                     3      2
      - 6y  - 24x + 48y, - 18x  - 27x y -
      -----------------------------------
          2      2              2
      9x*y  + 30x  + 24x*y + 12y  + 12x - 24y)
      ---------------------------------------
o12 : Ideal of R
```

The ring defined can have variable ordering based on degree, lexicographical ordering, or reverse lexicographical or a combination of the above.

17.11.2 CoCoA

The CoCoA System (Computations on Commutative Algebra) is an open-source mathematical system for performing research and computation in commutative algebra. CoCoA is written around a C++ library CoCoALib. The text console for CoCoA is shown below, and the Qt GUI frontend is shown in Figure 17.9.

```
------------------------------------------------------
---            ___/        ___/          \            ---
--            /     _ \   /     _ \     , \            --
--            \     |   | \     |   |  ___ \            --
---           ____, __/   ____, __/ _/    _\           ---
------------------------------------------------------
--     Version      : 4.7.4                           --
--     Online Help  : type  ?   or   ?keyword         --
--     Web site     : http://cocoa.dima.unige.it      --
------------------------------------------------------

------------------------------
-- The current ring is R ::= QQ[x,y,z];
```

```
------------------------------
```

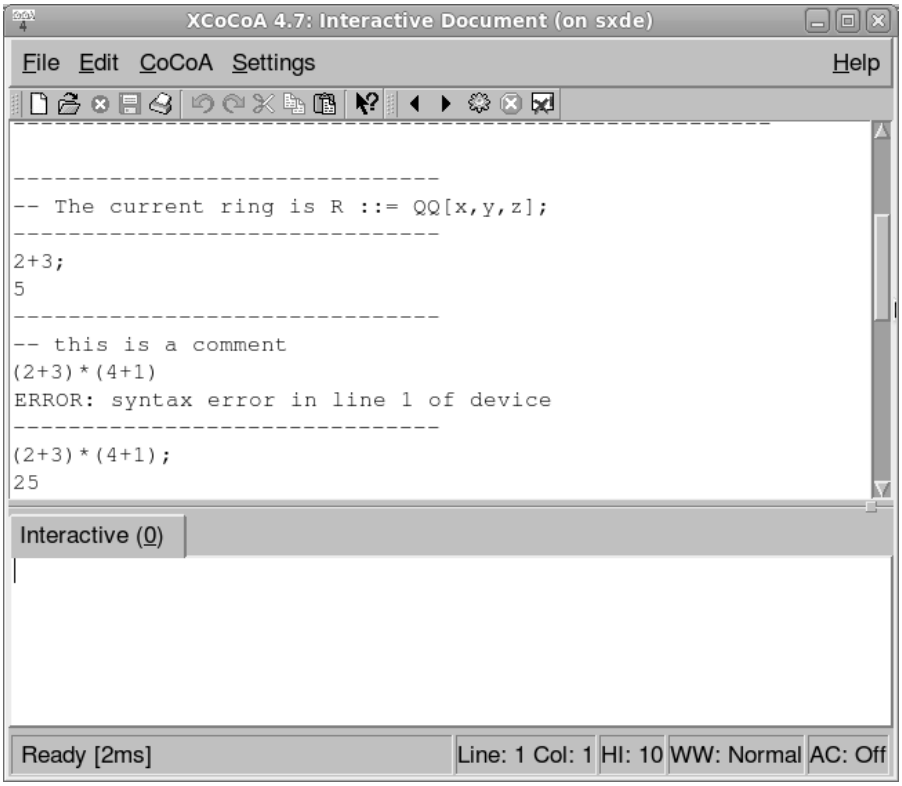


Fig. 17.9 xcocoa : GUI frontend for CoCoA

Interacting with CoCoA using the text console is similar to the other math software described earlier in this chapter. Variables in CoCoA have to be defined starting with uppercase letters as:

```
A := 5;
5A;
25
------------------------------
Use S ::= Z/(5)[a,b,c];
F := a-b;
I := Ideal( F^2, c );
I;
Ideal(a^2 - 2ab + b^2, c)
```

Like Singular, CoCoA also needs a ring to operate in. In the above example we have a created a ring S which is defined as modulo 5 with variables a, b, c. In another example with modulo-5 ring:

```
Use S ::= Z/(5)[a,b,c];
```

```
F := (3*a + 2*b);
F;
-2a + 2b
------------------------------
F*F
;
-a^2 + 2ab - b^2
------------------------------
F^5;
-2a^5 + 2b^5
```

We define a polynomial, and then compute the ideal of another polynomial. Groebner basis computations can also be carried out in such rings. Consider the following example of elimination:

```
------------------------------
Use R ::= Q[t,x,y];
GBasis( Ideal( t^3+3*t-x, 2*t^2+y));
[
  2t^2 + y,
  1/2ty - 3t + x,
  -2tx + 1/2y^2 - 3y,
  1/4y^3 + 2x^2 - 3y^2 + 9y]
Elim(t, Ideal(t^3+3*t-x, 2*t^2+y));
Ideal(1/4y^3 + 2x^2 - 3y^2 + 9y)
```

Variable substitution can be performed as:

```
Use R ::= Q[x,y,z];
F := (x+y+z)*(x+1);
Eval(F,[1]);
2y + 2z + 2
------------------------------
Subst(F,z,0);
x^2 + xy + x + y
------------------------------
```

17.12 CGAL (Computer Geometry Algorithms and Library)

CGAL is an open-source C++ library which aims to provide data structures and algorithms to solve computational geometry problem such as like triangulations, Voronoi diagrams, Boolean operations on polygons and on polyhedra, arrangements of curves, mesh generation, geometry processing, and convex hull algorithms.

Consider the following example using CGAL as shown in Listing 17.2.

```
// \file chull_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Example use of CGAL library
#include <iostream>           // for Program IO
#include <cassert>            // assertion checking
#include <vector>             // std::vector of points
#include <CGAL/Exact_predicates_inexact_constructions_kernel.h>
#include <CGAL/convex_hull_2.h>

typedef CGAL::Exact_predicates_inexact_constructions_kernel K;
typedef K::Point_2 Point_2;
typedef std::vector<Point_2> Point_Vector;

int main() {
  Point_Vector inputP, outputP;
  inputP.push_back( Point_2(0,0 ) );
  inputP.push_back( Point_2(0,5 ) );
  inputP.push_back( Point_2(2,5 ) );
  inputP.push_back( Point_2(2,7 ) );
  inputP.push_back( Point_2(7,7 ) );
  inputP.push_back( Point_2(7,0 ) );
  inputP.push_back( Point_2(3,0 ) );

  CGAL::convex_hull_2( inputP.begin(), inputP.end(),
            std::back_inserter( outputP ) );
  std::cout << "Convex hull has " << outputP.size() << " points.\n";
  std::copy( outputP.begin(), outputP.end(),
       std::ostream_iterator<Point_2>( std::cout, " " ) );
  std::cout << std::endl;
  return(0);
}
```

Listing 17.2 Example of using CGAl to compute convex hull

We can compile and run this program as follows:

```
$g++ -I$CGAL/include chull_example.cpp
     -l CGAL_Core -l CGAL
$ ./a.out
Convex hull has 4 points.
0 0 7 0 7 7 2 7
```

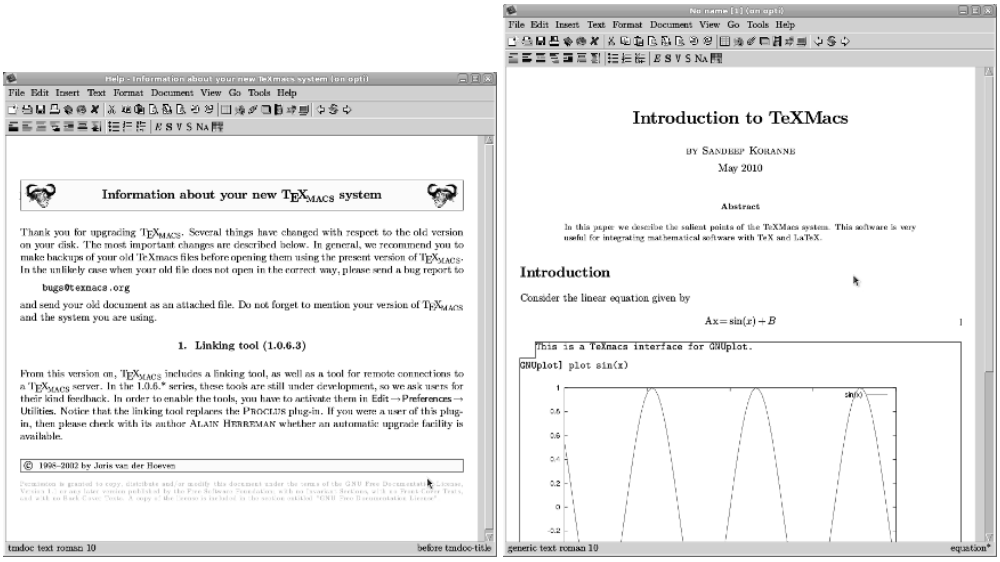
CGAL Library has the following functionality:

1. Geometry Kernels: as we saw in the above example, CGAL has inexact and exact computation kernels. Many algorithms need exact computation to arrive at the correct answer in the presence of degenerate input. CGAL's design allows the use of inexact kernels where the algorithm can work around the loss of precision,
2. Combinatoric algorithms: CGAL supports monotone and sorted matrix search, linear and quadratic programming; these operations are available as standalone, but usually they are used in a geometric setting to implement a higher level algorithm,
3. Convex hull: above we have seen an example of 2d convex hull. CGAL has convex hull algorithms for 2d, 3d and dD, including Delaunay Triangulations. Voronoi Diagrams are also included.
4. Polygons and Nef Polygons: CGAL supports operations, including generalized Booleans on polygons, collections of polygons and Nef polygons. Moreover, CGAL also supportes operations on Nef polygons embedded on the sphere. It

also has functions for polygon partitioning, straight skeleton and polygon offsetting. Minkowski sums of polygons can also be computed,

5. Operations on Polyhedra: CGAL includes the *half-edge* data structure for polyhedra representation and supports 3d boolean operations on Nef polyhedra,
6. 2d Arrangements: arrangements are combinatoric structures representing a collection of intersecting lines in the plane (for 2d, generalized to any dimension). 2d intersection and arrangement of curves is also supported. CGAL also includes an implementation of *snap rounding* to represent arrangements in finite precision,
7. Mesh Generation: CGAL includes mesh generation tools and algorithms to generate conforming triangulations, and surface reconstruction from points,
8. Spatial Searching and Sorting: CGAL includes the interval skip list data structure, and implements a 2d range search and neighbor search algorithm. It also includes an implementation of dD range trees, segment trees, and AABB trees. Rectangle intersection (and its generalization to dD iso-oriented box intersection) is also implemented.

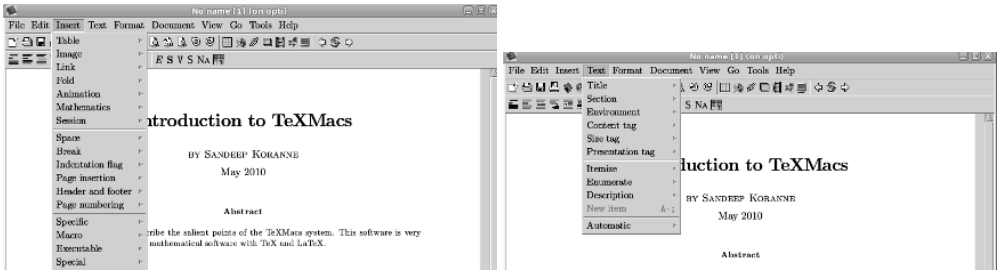
17.13 TeXMacs



(a) Example of TeXMacs

Fig. 17.10 TeXMacs software

GNU TeXmacs is an open-source WYSIWYG (what you see is what you get) style editing platform with special features for scientists. It can render equations and mathematics formulas, an example is shown in Figure 17.10(a) and (b).



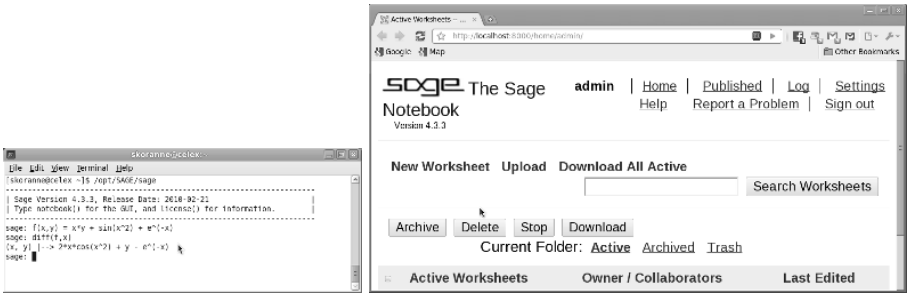
(a) More example of TeXMacs

Fig. 17.11 TeXMacs software calling external math programs

Texmacs is also integrated with mathematics software such as GNU Octave and Maxima. Example is shown in Figure 17.11 (a) and (b).

17.14 Sage

In this chapter we have seen many mathematics software, each having its own syntax for defining variables, polynomials, matrices and solving equations. Sage is an open-source mathematics software system which combines many of the above software system using a common Python based interface. Sage includes a GUI which executes in a browser, as well as a text interface. These are shown in Figure 17.12 and Figure 17.13.



(a) SAGE software TEXT and notebook

Fig. 17.12 SAGE mathematical software

It is also possible to try out Sage online before installing it, however, we greatly recommend the software if you need access to multiple math software systems using a common interface. Consider the simple example of solving a quadratic equation in x:

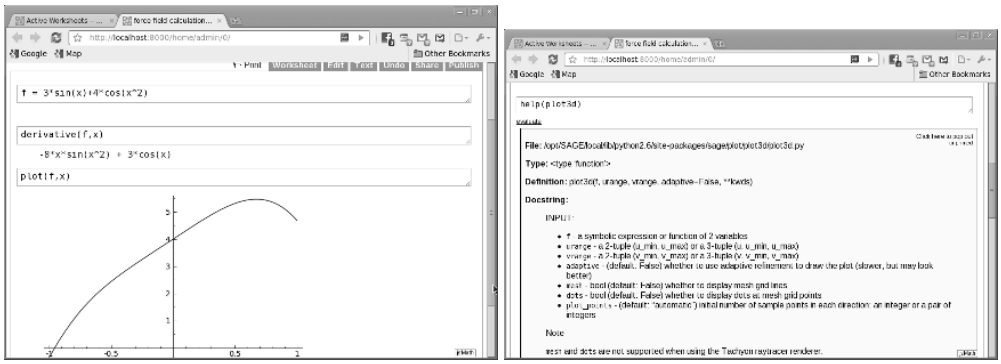
```
$/opt/SAGE/sage
```

```
----------------------------------------------------------
| Sage Version 4.3.3, Release Date: 2010-02-21
| Type notebook() for GUI, and license() for information.
----------------------------------------------------------
sage: x = var('x')
sage: solve(x^2-6*x+8,x)
[x == 4, x == 2]
sage: diff(cos(x^2),x,4)
16*x^4*cos(x^2) + 48*x^2*sin(x^2) - 12*cos(x^2)
```

The second example computes the 4th differential of the expression $\cos x^2$. Sage can also perform integration, and solving partial differential equations. Internally, Sage is connected to many of the mathematical software we have discussed in this chapter. For example, Sage can connect to PARI to compute the 'chebyshev_U' function:

```
sage: y = polygen(QQ,'y')
sage: che
chebyshev_T  chebyshev_U  checkbox
sage: cheby
chebyshev_T  chebyshev_U
sage: chebyshev_U(2,y)
4*y^2 - 1
sage: bessel_I( 1,1, "pari", 100)
0.56515910399248502720769602761
```

Sage supports TAB-completion, to complete names of functions as shown in the above example.



(a) SAGE notebook and help screen

Fig. 17.13 SAGE mathematical software

As shown in Figure 17.13, Sage can be executed on a more powerful computer and then multiple remote users can connect to the Sage software using a browser. The *notebooks* which represent the computation in Sage are persisted on the Sage

server and can be used to perform computation. This greatly reduces the administrative overhead with maintaining Sage for a group of scientists. To invoke a specific math software that is integrated with Sage, we can use the following form:

```
sage: maxima.eval("diff(sin(x),x)")
'cos(x)'
```

Thus, Sage can be used a single, homogeneous software system which can be used to perform computations in any one of the software, and also to use a combination to solve a problem.

17.15 Conclusion

In addition to mathematical libraries, there exists a number of open-source mathematical software systems which implement complete mathematical framework. These are described in this chapter, and include Maxima, GNU Octave, R, PSPP, Pari/GP, Nauty, OpenAxiom, Reduce, Singular, and polymake. Other specialized math software for algebraic geometry include Macaulay2 and CoCoA. The CGAL (computer geometry algorithms library) implements many computational geometry algorithms and data-structures. Mathematical front-ends to the above mentioned software include TeXMacs and SAGE. SAGE, in particular combines many of the features of the previously described software with uniform notation and the ability to pass data from one tool to another.